Exception Handling

Topic 8 – Exception handling

File I/O

Intended Learning Outcomes

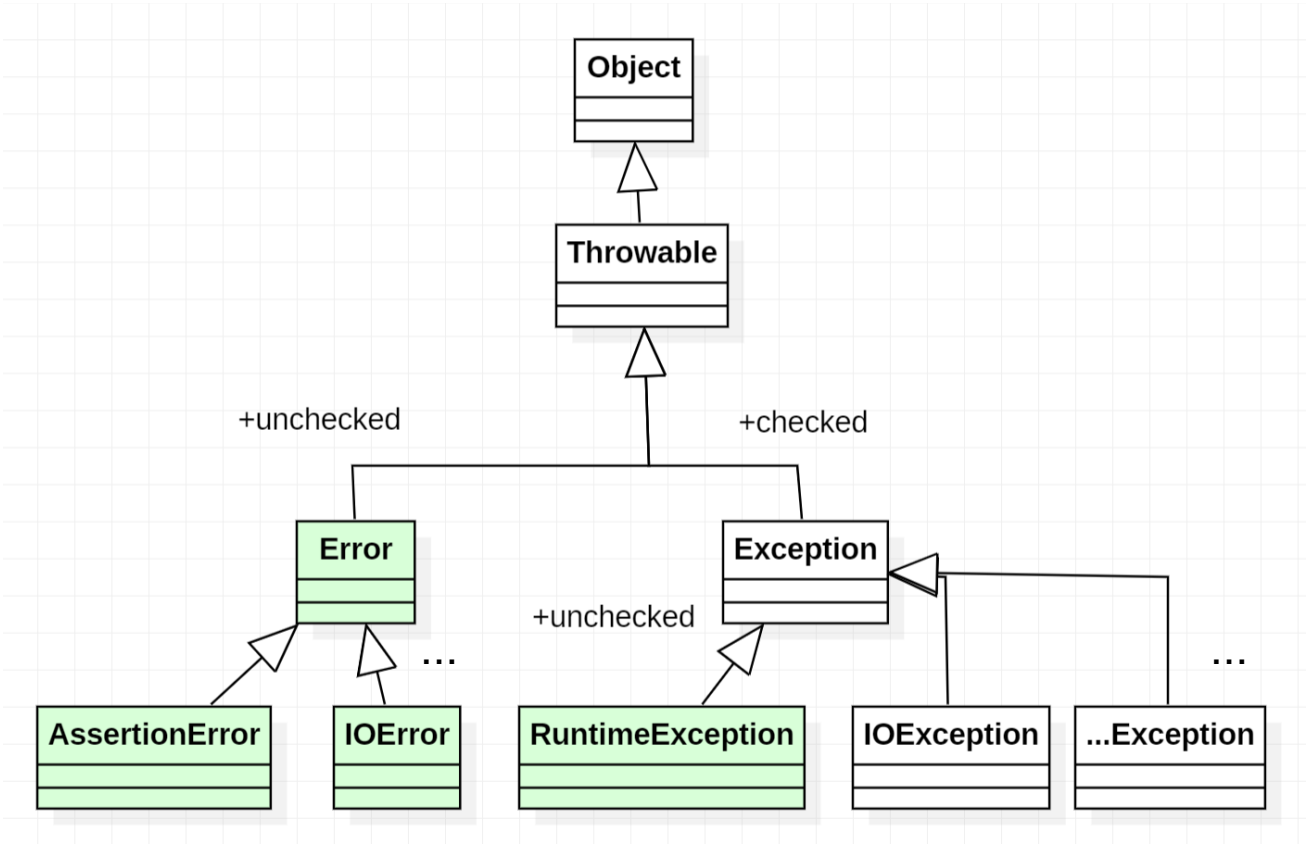
- Describe the exception handling mechanism in Java
- Distinguish between different types of exceptions
- Apply different exception handling approaches in Java

Usage of Exception Handling

- Used to manage and handle errors or exceptional situations during execution
- Handling Errors – e.g. dividing by zero, accessing array elements out of bounds, or encountering null references.
- Robustness - prevent from crashing
- Resource Management - using try-with-resources
  - ensure that resources are properly closed and released.
- Fault Tolerance

Exception Handler

- Code that gracefully responds to unexpected situation
- Default exception handler
  - Deal with unhandled exception
  - Print error message and crash



Throwable, Error and Exception

https://docs.oracle.com/javase/8/docs/api/?java/lang/Error.html

Exception classes

- Exception & Error – derived from Throwable class
- Error
  - Critical Run-time error e.g. JVM, out of memory
  - Application should not try to handle
- Exception
  - Programmers should handle

Syntax of Exception Handling

```
try{
    // code to try running
}catch (ExceptionType exName){
    // Specific Exception type to catch
}catch (ExceptionType2 exName){
    // code to handle generic Exception
}finally{
    // code that always runs
}
```

try block

```
try{
    // code to try running
}catch …
```

- Statements to execute
- Potentially throw exception
- Application does not halt if exception throwed
- catch clauses follow

catch clause

```
catch (ExceptionType parameterName){
 // catch block
}
```

- With exception, control is transferred to the matched catch block
- Multiple catch blocks can be used
- handle different types of exceptions.

`finally` block (optional)

```
}finally{
    // code that always runs
}
```

- Optional clause
- must follow try-catch clauses
- execute code that should always run
  - exception occurred or not.
- ensures that resources are properly cleaned up

try-catch-finally Example

```java
public class TryCatchFinallyExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[4]); // Trying to access an index out of bounds
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("An ArrayIndexOutOfBoundsException occurred: " + e.getMessage());
        } finally {
            System.out.println("Finally block is always executed, regardless of exceptions.");
        }
    }
}
```

Other details

- Exception Type
  - To Catch an Exception, Exception Type must be compatible with the thrown type
- Flow after catch
  - Program continue if successfully catch Exception
- Use inherited Throwable.getMessage() to get detail message

https://www.javatpoint.com/post/java-throwable-getmessage-method

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Throwable.html#getMessage()

Multiple Exception Handling

- Try block may throw more than 1 type of exception
- catch clause needs to be written for each type of exception that could potentially be thrown.
- JVM will run the first compatible catch clause found.
- catch clauses must be listed from most specific to most general.

Polymorphic Reference

- first compatible type would catch the Exception
- catching generic Exception type
  - catch any exception derived from Exception class
  - order matters
- See: `PolymorphicExceptionHandlingExample.java`

PolymorphicExceptionHandlingExample.java

```java
public class PolymorphicExceptionHandlingExample {
    public static void main(String[] args) {
        try {
            // Simulating an exception by dividing by zero
            int result = 5 / 0;
            System.out.println(result);
        } catch (ArithmeticException ex) {
            System.out.println("ArithmeticException caught: " + ex.getMessage());
        } catch (Exception ex) {
            System.out.println("Exception caught: " + ex.getMessage());
        }
    }
}
```

Multi-Catch Example

- Specify more than one exception in a catch
  - Since Java 7

```
try {
    // Additional code that may throw exceptions
} catch (IOException | SQLException e) {
    System.out.println("An I/O or SQL exception occurred: " + e.getMessage());
}
```

Stack Trace

- a list of all the methods in the call stack.
  - method that was executing when an exception occurred
  - all of the methods that were called
- Call stack
  - an internal list of all the methods that are currently executing.
- See StackTraceExample.java

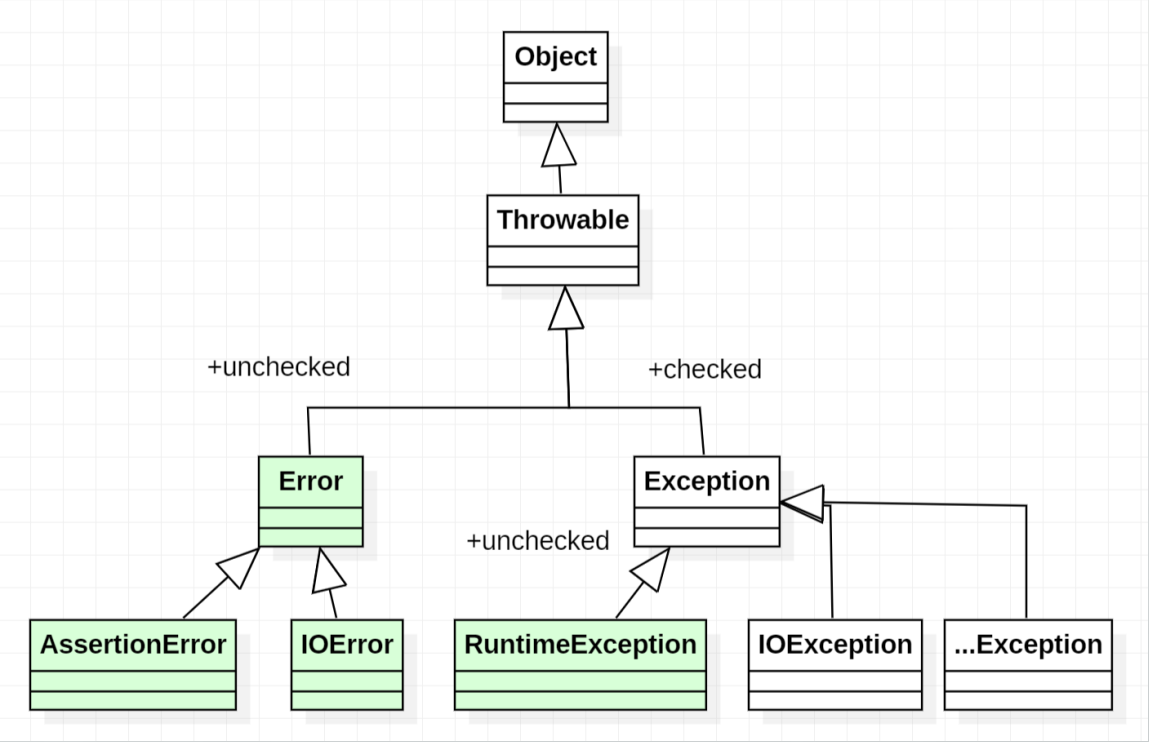
```
java.lang.ArithmeticException: / by zero
        at StackTraceExample.method2(StackTraceExample.java:14)
        at StackTraceExample.method1(StackTraceExample.java:10)
        at StackTraceExample.main(StackTraceExample.java:4)
```

Bubbling of Uncaught Exceptions

- "bubbles up" in the call stack.
  - When an exception is thrown but not caught within a method
- exception propagates up through the method calls until
  - it encounters a suitable catch block or
  - reaches the top level of the call stack.
- Default exception handling
  - program terminates
  - exception details are typically printed to the console

Checked and Unchecked Exceptions

- Unchecked exceptions
  - derived from the Error class or
  - the RuntimeException class.
- Checked exceptions
  - exceptions that are NOT
    - derived from Error or RuntimeException
  - Compiler would check them



Unchecked Exceptions

- Exceptions derived from Error
  - thrown when a critical error occurs and should not be handled.
- RuntimeException
  - superclass for exceptions that result from programming errors.
  - exceptions can be avoided with properly written code.
- In most cases, should not be handled.

Handling of Checked Exceptions

- If the code in a method can throw a checked exception, the method:
  - must handle the exception, or
  - it must have a throws clause listed in the method header.
- `throws clause`
  - informs the compiler what exceptions can be thrown from a method.

Checked and Unchecked Exceptions

```
public static void main(String[] args) throws
FileNotFoundException {
    File file = new File("example.txt");
    Scanner scanner = new Scanner(file);
    while (scanner.hasNextLine()) {
      System.out.println(scanner.nextLine());
    }
    scanner.close();
} // main()
```

Throwing Exceptions

- You can write code that:
  - throws one of the standard Java exceptions, or
  - an instance of a custom exception class that you have designed.
- throw statement
  - used to manually throw an exception.

```
throw new ExceptionType(MessageString);
```

Throwing Exceptions

- MessageString argument
  - contains a custom error message that can be retrieved from getMessage method.
- If you do not pass a message to the constructor, the exception will have a null message.
  - throw new Exception("Out of fuel");
  - Note: Don't confuse the throw statement with the throws clause.
- Example: DieExceptionDemo.java

Creating Exception Classes

- create custom exception classes
  - by deriving them from the `Exception` class or one of its derived classes.
- Example:

```
public class NegativeStartingBalance
extends Exception
```

- Details:
  - BankAccount.java, NegativeStartingBalance.java, AccountTest.java

`@exception` Tag in JavaDoc

- General format: `@exception ExceptionName Description`
- Example:

```
@exception NegativeStartingBalance Exception to
throw …
```

- Rules
  - @exception tag in a method's documentation comment must appear after the general description of the method.
  - The description can span several lines. It ends at the end of the documentation comment (the */ symbol) or at the beginning of another tag.

File Input/Output

Reading Files in Java

- `File` Class:
  - Provides information about files and directories (existence, size, etc.),
  - but doesn't directly read content. Used to create `Scanner` objects.
- `Scanner` Class:
  - Simplifies reading various data types (integers, strings, etc.) from different sources, including files.
  - Handles whitespace automatically.

Workflow of reading file

1. Create a `File` object representing the file to read.
2. Create a `Scanner` object, passing the `File` object to its constructor.
3. Use `Scanner` methods (e.g., `nextInt()`, `nextLine()`, `hasNextInt()`) to read data.
4. Handle potential `FileNotFoundException` (using `try-catch`).
5. Close the `Scanner` (implicitly handled by try-with-resources).

File Scanner Example

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileScannerExample {
public static void main(String[] args) {
    File file = new File("data.txt");
    try (Scanner scanner = new Scanner(file)) {
        while (scanner.hasNextInt()) {
            int num = scanner.nextInt();
            // Process the integer
            System.out.println("Read:" + num);
        }
    } catch (FileNotFoundException e) {
        System.err.println("File not found: " + e.getMessage());
    }
}
}
```

File output

- Purpose: Write data to files on the file system.
- Core Classes:
  - `FileOutputStream` (bytes),
  - `FileWriter` (characters),
  - `BufferedWriter` (buffered characters).
- Error Handling: `IOExceptions` must be handled using `try-catch` blocks.
- `BufferedWriter` offers significantly improved performance over direct file writing.
- Data Types: Can handle various data types, often converting them to bytes or characters before writing.

BufferedWriterExample.java

```
String filePath = "output.txt";
String[] lines = {"This is the first line.", "This is the second line.",
"This is the third line."};
try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath)))
{
   for (String line : lines) {
       writer.write(line);
       writer.newLine(); // Adds a newline character
   }
   System.out.println("Data written to file: " + filePath);
} catch (IOException e) {
    System.err.println("An error occurred: " + e.getMessage());
}
```

Object serialization

- How to save/load an object to/from a file?
- Serialization:
  - Turns an object into a stream of bytes (like a digital copy).
  - Java handles the details;
  - we just need to mark our class as `Serializable`.

Serializable

- Add `implements Serializable` to your class declaration.
  - That's it! (mostly - see next point)
- What's NOT Saved:
  - `static` variables (belong to the class, not the object)
  - `transient` variables (marked with `transient` keyword)

Serialization Example

```
class Person implements Serializable {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() { return name; }
    public int getAge() {  return age; }
    @Override
    public String toString() {
        return "Person{" + "name='" + name + '\'' +
                ", age=" + age + '}';
    }
}
```

Saving Object (Serialization):

- Create an `ObjectOutputStream`.
- Use `writeObject()` to save the object to a file.

Serialization Example - Saving

```
Person person = new Person("Alice", 30);
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("person.dat")))
{
    oos.writeObject(person);
    System.out.println("Object serialized successfully.");
} catch (IOException e) {
    System.err.println("Error serializing object: " + e.getMessage());
}
```

Loading (Deserialization):

- Create an `ObjectInputStream`.
- Use `readObject()` to load the object from a file.
- Cast the result back to your object's type.

Serialization Example – Loading

```
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("person.dat")))
{
    Person loadedPerson = (Person) ois.readObject();
    System.out.println("Object deserialized successfully: " + loadedPerson);
} catch (IOException | ClassNotFoundException e) {
    System.err.println("Error deserializing object: " + e.getMessage());
}
```

Summary

Exception class

try-catch-finally

File I/O